

SCHOOL OF COMPUTER SCIENCES
UNIVERSITI SAINS MALAYSIA

CPC353
NATURAL LANGUAGE PROCESSING

Academic Session 2024/2025
Semester 1

Assignment 2 Report

Lecturer:
Assoc. Prof. Dr. Tan Tien Ping

Group Member:

NAME	MATRIC NUMBER	EMAIL
NADIATUL FAKHIRA BINTI ZAKPAR	160307	nadiatul123@student.usm.my
NORITA BINTI MUIN	160453	noritamuin@student.usm.my

Submission Date:
27th January 2025

TABLE OF CONTENT

1.0 INTRODUCTION....2
2.0 FEATURE EXTRACTION....3
3.0 DATA PREPROCESSING....6
3.1 Assigning label speaker changes....6
3.2 Normalization of numeric features....8
4.0 MODEL DEVELOPMENT....12
4.2 Handling imbalanced dataset....14
4.3 Experiment with another dataset of different window size....15
5.0 RESULT AND DISCUSSION....16
5.1 Test the model on unseen data....16
5.2 Comparison performance on test data between 3 model of different window sizes with balanced or imbalanced dataset....17
5.3 Analyze model performance at detecting speaker change points, discussing strengths and weaknesses. suggestions for improvement....23
6.0 CONCLUSION....26
7.0 REFERENCES....27

1.0 INTRODUCTION

Speaker change detection is a critical task in conversation analysis. This report outlines the steps in developing solutions by implementing the combination of natural language processing and deep learning model. We explore speaker change detection using the IEMOCAP dataset. IEMOCAP is a widely recognized resource for studying conversations, emotions, and speaker interactions, making it ideal for speaker change detection research. This dataset is organized into folders containing recordings from five sessions and an additional reference folder. Each session includes a dialogue folder with audio files in *.wav* format, where two actors engage in scripted or improvised conversations. The reference folder contains annotations, such as *RTTM* files, which provide detailed information about speaker turns, timings, and other metadata. These annotations are essential for identifying speaker changes, as they act as the ground truth for training and evaluating models.

Speaker change detection involves identifying moments when one speaker stops, and another begins, which is useful for applications like transcription and emotion analysis. To build a detection system, audio features such as Mel-Frequency Cepstral Coefficients (MFCCs), pitch, and intensity are extracted from the `.wav` files. These features capture voice characteristics and help models distinguish speakers. By aligning these features with the timing data in the RTTM files from the reference folder, models can be trained to detect transitions accurately.

Despite its advantages, working with IEMOCAP presents challenges like overlapping speech and subtle transitions between speakers. Advanced techniques, including deep learning models like Recurrent Neural Networks (RNNs) or Transformers, can address these issues by analyzing temporal patterns in the audio. Overall, the IEMOCAP dataset provides a robust foundation for developing speaker change detection models, which play a critical role in improving conversational analysis and related technologies.

2.0 FEATURE EXTRACTION

This project began by extracting features such as Fundamental Frequency (F0) and Mel-Frequency Cepstral Coefficients (MFCC). F0 represents pitch while MFCC captures 13 spectral properties of speech. We performed two experiments by changing the frame size and frame length. The experiment 1 was conducted by setting up a sliding window with 20ms frames and 10ms overlap. In the experiment 2, we employed another different configuration which is 80ms sized frames with 10ms overlap. The feature extraction process is implemented using Python libraries such as librosa and praat-parselmouth.

```
def extract_f0(audio_path, frame_length=0.020, frame_shift=0.010):
    sound = parselmouth.Sound(audio_path)
    pitch = sound.to_pitch(time_step=frame_shift, pitch_floor=75, pitch_ceiling=500)
    f0 = pitch.selected_array['frequency']
    return f0
```

Figure 2.1 Function created to extract f0

Figure 2.1 displays the function *extract_f0* which is responsible for extracting the fundamental frequency (F0) from the audio file. This function takes an *audio_path* parameter, which is the file path to the audio file, and two optional parameters *frame_length* and *frame_shift* that control the frame length and frameshift used in the pitch extraction process.

```
def extract_mfcc(audio_path, frame_length=0.020, frame_shift=0.010, n_mfcc=13):

    # Load audio file
    y, sr = librosa.load(audio_path)

    # Convert frame length and shift from seconds to samples
    frame_length_samples = int(frame_length * sr)
    hop_length_samples = int(frame_shift * sr)

    # Extract MFCCs
    mfccs = librosa.feature.mfcc(
        y=y,
        sr=sr,
        n_mfcc=n_mfcc,
        win_length=frame_length_samples,
        hop_length=hop_length_samples
    )

    return mfccs.T
```

Figure 2.2 Function created to extract the MFCC value

Figure 2.2 shows the function *extract_mfcc* which is used to extract the Mel-Frequency Cepstral Coefficients (MFCCs) from an audio file. This function takes an *audio_path* parameter, which is the file path to the audio file, and three optional parameters which are *frame_length*,

frame_shift and *n_fcc*. These optional parameters control the frame length, frame shift, and the number of MFCC coefficients to be extracted, respectively. The function uses the librosa library to load the audio file, converting the frame length and shift from seconds to samples. The *librosa.feature.mfcc* function is then called to extract the MFCC features from the audio signal. The extracted features are saved in a csv file in the root folder in the csv format by sessions.

	file_name	frame_index	start_time	end_time	f0	mfcc_1	\
0	Ses01F_impro01.wav	0	0.00	0.02	0.0	-105.007416	
1	Ses01F_impro01.wav	1	0.01	0.03	0.0	-195.412380	
2	Ses01F_impro01.wav	2	0.02	0.04	0.0	-219.190200	
3	Ses01F_impro01.wav	3	0.03	0.05	0.0	-226.481160	
4	Ses01F_impro01.wav	4	0.04	0.06	0.0	-227.054210	
5	Ses01F_impro01.wav	5	0.05	0.07	0.0	-259.877660	
6	Ses01F_impro01.wav	6	0.06	0.08	0.0	-276.590400	
7	Ses01F_impro01.wav	7	0.07	0.09	0.0	-288.580050	
8	Ses01F_impro01.wav	8	0.08	0.10	0.0	-294.471400	
9	Ses01F_impro01.wav	9	0.09	0.11	0.0	-297.975400	

	mfcc_11	mfcc_12	mfcc_13
..	10.969068	-28.467300	-13.787360
	17.197315	-9.572943	-2.200324
	20.728607	-8.207458	18.923134
	2.085654	-30.863335	-1.439828
	21.499125	-13.687496	11.752371
	12.878948	-6.407865	19.718824
	8.901363	-17.487396	5.872521
	16.711110	-11.930466	5.216793
	-0.217814	-18.171019	10.560537
	10.356131	-20.535637	-16.681473

Figure 2.3 Top 10 rows of features1_session1.csv.

Figure 2.3 displayed the top 10 rows of the extracted feature values for the audio file "Ses01F_impro01.wav". The columns include the file name, frame index, start time, end time, F0 (fundamental frequency), and the 13 MFCC (Mel-Frequency Cepstral Coefficient) values that were extracted.

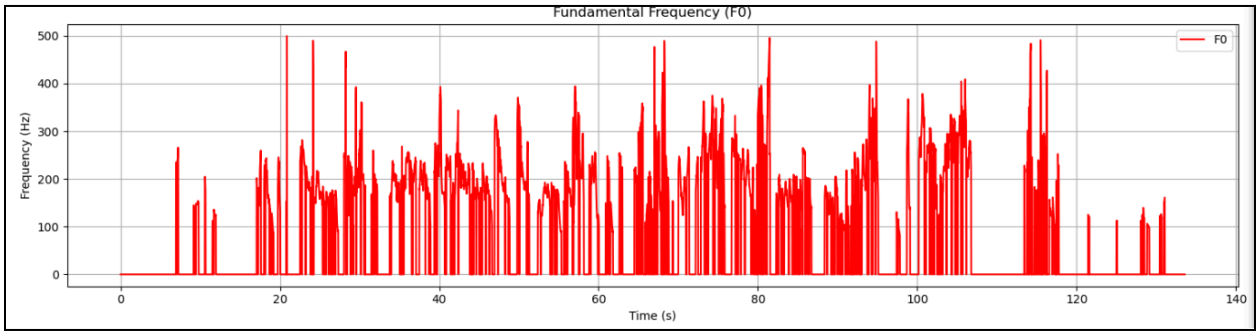


Figure 2.4 Fundamental Frequency (F0) over time

Figure 2.4 displays the graph for fundamental frequency (F0) values over time for the same audio file. The x-axis represents time in seconds, and the y-axis shows the F0 values in Hz. The plot displays the fluctuations in the fundamental frequency throughout the duration of the audio.

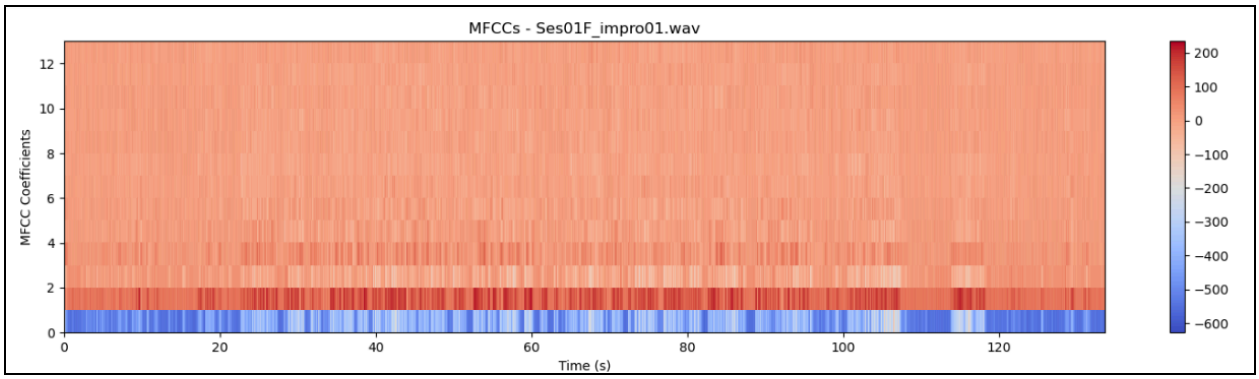


Figure 2.5 Mel-Frequency Cepstral Coefficients (MFCC) over time

Figure 2.5 shows the Mel-Frequency Cepstral Coefficients (MFCCs) for the audio file "Ses01F_impro01.wav" over time. MFCCs are a widely used feature in speech processing and music analysis. The x-axis represents the time in seconds, while the y-axis shows the MFCC coefficients. The image has a heat map color representation, with higher values in red and lower values in blue. This allows to visualize the patterns and changes in the MFCC features over the duration of the audio file.The graph provides the spectral characteristics of the audio signal, where the MFCC patterns can provide insights into the underlying audio content and its evolution over time.

3.0 DATA PREPROCESSING

In this section, we will explain the process of how we do the data preprocessing steps. It involved assigning label data for speaker changes (1 for change and 0 for no change). Then, we also handle normalization for numerical features and lastly, splitting the dataset into training, validation and testing set to prepare for modelling.

3.1 Assigning label speaker changes

```python
def read_rttm_file(rttm_path):
    """
    Read RTTM file and return a list of speaker segments
    """
    speaker_segments = []
    with open(rttm_path, 'r') as f:
        for line in f:
            parts = line.strip().split()
            if parts[0] == 'SPEAKER':
                speaker_segments.append({
                    'file': parts[1],
                    'time_start': float(parts[3]),
                    'duration': float(parts[4]),
                    'end_time': float(parts[3]) + float(parts[4]),
                })
    return speaker_segments
```

Figure 3.1 Function to read RTTM files and return a list of speaker segments

The assigning label process is mainly handled by getting data from RTTM files from the reference folder to best suit the label of speaker_change for each row. The process begins with reading RTTM files, which contain speaker segment information. The **read_rttm_file()** function reads these files and extracts relevant details such as the file name, time start, duration, and end time of each speaker segment. This information is stored in a list of dictionaries for further processing.

```python
def label_speaker_changes(features_df, base_path):
    """
    Label speaker changes based on matching RTTM files
    """
    # Create a copy of the DataFrame to avoid modifying the original
    labeled_df = features_df.copy()

    # Add speaker change label column
    labeled_df['speaker_change'] = 0

    # Find unique wav files
    unique_wav_files = features_df['file_name'].unique()

    for wav_file in unique_wav_files:
        # Remove .wav extension and find matching RTTM
        base_filename = os.path.splitext(wav_file)[0]

        # Search for RTTM file in the entire base path
        matching_rttm_files = []
        for root, dirs, files in os.walk(base_path):
            matching_rttm_files.extend([
                os.path.join(root, f) for f in files
                if base_filename in f and f.endswith('.rttm')
            ])

        if not matching_rttm_files:
            print(f"No RTTM file found for {wav_file}")
            continue

        # Use the first matching RTTM file
        rttm_path = matching_rttm_files[0]

        # Read speaker segments from RTTM file
        speaker_segments = read_rttm_file(rttm_path)

        # Label speaker changes
        for segment in speaker_segments:
            # Find rows within the speaker segment
            mask = (
                (labeled_df['file_name'] == wav_file) &
                (labeled_df['start_time'] < segment['time_start']) &
                (labeled_df['end_time'] >= segment['time_start'])
            )

            labeled_df.loc[mask, 'speaker_change'] = 1

    return labeled_df
```

Figure 3.2 Function to label speaker change

Next, the **label_speaker_changes() function** in Figure 3.2 labels speaker changes in the dataset. It starts by creating a copy of the original DataFrame to avoid modifying it directly. A new column, speaker_change, is added to the DataFrame and initialized to 0. The function then identifies unique WAV files in the dataset with the corresponding RTTM files. For each matching RTTM file, the function reads the speaker segments.

The core labeling mechanism involves by comparing the RTTM file's time_start against each row's start_time and end_time. When a segment's start time meets the criteria which is less than the row's start_time and greater than or equal to its end_time, that specific audio frame is labeled with a 1, indicating a speaker change. Frames not matching these exact temporal conditions receive a 0 label, signifying no speaker transition.

Below, we will show some output of labelling csv files with speaker_change 0 and 1 by reference from RTTM files.

```
SPEAKER Ses01F_impro01 1 6.290 1.946 <NA> <NA> Ses01F_impro01_F <NA> <NA>
SPEAKER Ses01F_impro01 1 7.571 2.904 <NA> <NA> Ses01F_impro01_M <NA> <NA>
```

Figure 3.3 Example of RTTM file from 'Ses01F_impro01.rttm'

Figure 3.3 showing a few samples of the start_time and duration of the speaker change from the *'Ses01F_impro01.rttm' file.* The labeling process refers to this file to know which line of rows would be labelled as 1 to detect speaker change. It will be on start time 6.290 and 7.571.

	file_name	frame_inde	start_time	end_time	speaker_change
2	Ses01F_impro01.wav	0	0	0.02	0
3	Ses01F_impro01.wav	1	0.01	0.03	0
4	Ses01F_impro01.wav	2	0.02	0.04	0
628	Ses01F_impro01.wav	626	6.26	6.28	0
629	Ses01F_impro01.wav	627	6.27	6.29	1
630	Ses01F_impro01.wav	628	6.28	6.3	1
631	Ses01F_impro01.wav	629	6.29	6.31	0
756	Ses01F_impro01.wav	754	7.54	7.56	0
757	Ses01F_impro01.wav	755	7.55	7.57	0
758	Ses01F_impro01.wav	756	7.56	7.58	1
759	Ses01F_impro01.wav	757	7.57	7.59	1
760	Ses01F_impro01.wav	758	7.58	7.6	0
761	Ses01F_impro01.wav	759	7.59	7.61	0

Figure 3.4 Labeled table from experiment with 20ms sliding window size recorded in 'labeled_features1_session1.csv'

	file_name	frame_inde	start_time	end_time	speaker_change
2	Ses01F_imp	0	0	0.08	0
3	Ses01F_imp	1	0.01	0.09	0
621	Ses01F_imp	619	6.19	6.27	0
622	Ses01F_imp	620	6.2	6.28	0
623	Ses01F_imp	621	6.21	6.29	1
624	Ses01F_imp	622	6.22	6.3	1
625	Ses01F_imp	623	6.23	6.31	1
626	Ses01F_imp	624	6.24	6.32	1
627	Ses01F_imp	625	6.25	6.33	1
628	Ses01F_imp	626	6.26	6.34	1
629	Ses01F_imp	627	6.27	6.35	1
630	Ses01F_imp	628	6.28	6.36	1
631	Ses01F_imp	629	6.29	6.37	0
632	Ses01F_imp	630	6.3	6.38	0
750	Ses01F_imp	748	7.48	7.56	0
751	Ses01F_imp	749	7.49	7.57	0
752	Ses01F_imp	750	7.5	7.58	1
753	Ses01F_imp	751	7.51	7.59	1
754	Ses01F_imp	752	7.52	7.6	1
755	Ses01F_imp	753	7.53	7.61	1
756	Ses01F_imp	754	7.54	7.62	1
757	Ses01F_imp	755	7.55	7.63	1
758	Ses01F_imp	756	7.56	7.64	1
759	Ses01F_imp	757	7.57	7.65	1

Figure 3.5 Labeled table from experiment with 80ms sliding window size recorded in 'labeled_features1_session2.csv'

From *'Ses01F_impro01.rttm'* as shown at figure 3.3 above recorded the first speaker change at 6.290. Thus window size that detects speaker changes were labelled to 1. For example, window sized 20ms in figure 3.4 ranging from '6.26 to 6.28' and '6.28 to 6.29' labelled to 1 while in figure 3.5 window that ranged '6.21 to 6.29' at row 623 labelled 1 until window ranged '6.28 to 6.36' at row 630. However, row 631 in both figure 3.4 and figure 3.5 labelled as 0 because there is no speaker changing phase even if the new speaker is currently speaking at this point.

3.2 Normalization of numeric features

```
def normalize_features_zscore(labeled_df):
    """
    Normalize numerical features using Z-score normalization
    """
    # Select numerical feature columns
    feature_columns = [col for col in labeled_df.columns if col.startswith('mfcc') or col == 'f0']

    # Initialize StandardScaler
    scaler = StandardScaler()

    # Apply Z-score normalization
    labeled_df[feature_columns] = scaler.fit_transform(labeled_df[feature_columns])

    print("Z-score normalization applied to numerical features.")
    return labeled_df
```

Figure 3.3 Function to normalize numerical F0 and MFCC features

We also handled normalization on numerical features using the normalize_features_zscore function, which applies Z-score standardization to fundamental frequency (F0) feature and all MFCC features. This is because we observed FO and MFCC features have different scales and ranges. By centering features around zero and scaling them to unit variance, the function ensures that no single feature dominates the model's learning. This normalization prevents potential biases introduced by varying feature scales and improves the performance of the model, especially for speaker change classification tasks.

Lastly, the **process_all_sessions() function** handles the entire preprocessing workflow. It identifies feature CSV files in the current directory and processes each file by labeling speaker changes and normalizing numerical features. The processed DataFrame is then saved to a new CSV file for example labelled_features_session1.csv. The function also provides a summary of the speaker change label distribution and displays the top 10 rows of the labeled DataFrame.

```python
def process_all_sessions(base_path):
    """
    Process all sessions, label speaker changes, and normalize features
    """
    # Find feature CSV files
    feature_files = [f for f in os.listdir('.') if f.startswith('features1_') and f.endswith('.csv')]

    for features_csv in feature_files:
        # Load features
        features_df = pd.read_csv(features_csv)

        # Label speaker changes
        labeled_df = label_speaker_changes(features_df, base_path)

        # Normalize numerical features
        labeled_df = normalize_features_zscore(labeled_df)

        # Save labeled and normalized features
        output_csv = f'labeled1_{features_csv}'
        labeled_df.to_csv(output_csv, index=False)

        print(f"Labeled and normalized features saved to {output_csv}")

        # Display labeling summary
        speaker_change_counts = labeled_df['speaker_change'].value_counts()
        print("\nSpeaker Change Label Distribution:")
        print(speaker_change_counts)
        print(f"Percentage of speaker change frames: {speaker_change_counts[1] / len(labeled_df) * 100:.2f}%\n")

        # Display top 10 rows of the labeled and normalized CSV
        print("Top 10 rows of the labeled and normalized CSV:")
        print(labeled_df.head(10))

def main():
    base_path = input("Enter the path to your IEMOCAP dataset: ")
    process_all_sessions(base_path)

if __name__ == "__main__":
    main()
```

Figure 3.4 Function to process all session 1 to session 5

```
Enter the path to your IEMOCAP dataset: C:\Users\User\Downloads\assignment2 cpc353\assignment2\reference
Z-score normalization applied to numerical features.
Labeled and normalized features saved to labeled_features1_session1.csv

Speaker Change Label Distribution:
0    817073
1      7559
Name: speaker_change, dtype: int64
Percentage of speaker change frames: 0.92%
```

```
Top 10 rows of the labeled and normalized CSV:
            file_name  frame_index  start_time  end_time        f0    mfcc_1  \
0  Ses01F_impro01.wav            0        0.00      0.05 -0.651306  3.535591
1  Ses01F_impro01.wav            1        0.01      0.06 -0.651306  3.437276
2  Ses01F_impro01.wav            2        0.02      0.07 -0.651306  2.844247
3  Ses01F_impro01.wav            3        0.03      0.08 -0.651306  2.642039
4  Ses01F_impro01.wav            4        0.04      0.09 -0.651306  2.516589
5  Ses01F_impro01.wav            5        0.05      0.10 -0.651306  2.280618
6  Ses01F_impro01.wav            6        0.06      0.11 -0.651306  2.067229
7  Ses01F_impro01.wav            7        0.07      0.12 -0.651306  1.912242
8  Ses01F_impro01.wav            8        0.08      0.13 -0.651306  1.837767
9  Ses01F_impro01.wav            9        0.09      0.14 -0.651306  1.813043

     mfcc_9    mfcc_10   mfcc_11   mfcc_12   mfcc_13  speaker_change
0 -0.445052 -1.733853  1.022961 -2.338850 -1.710983               0
1 -0.374395 -1.888261  1.108210 -1.473224 -1.199608               0
2 -0.158267 -1.502301  1.482068 -1.233890  0.619105               0
3  0.135722 -1.286181  1.378313 -1.932489  1.294693               0
4 -0.228808 -1.523478  1.221176 -1.987947  1.167619               0
5 -0.534431 -1.203212  1.317764 -1.292177  1.458964               0
6 -0.785568 -1.099102  0.953226 -1.620577  0.946780               0
7 -0.393512 -1.471028  1.105640 -1.821914  0.149807               0
8  0.310473 -1.698621  0.820159 -1.918406 -0.141931               0
9 -0.022282 -1.681909  0.431658 -2.488740 -0.959776               0
```

Figure 3.5 Output after assigning labels and doing normalization

- Divide the data into train (session 1-3), validation (session 4), and test (session 5) sets

```python
def combine_csv_files(output_train, output_val, output_test):
    """
    Combine CSV files into train, validation, and test datasets.

    """
    # Initialize empty lists to hold DataFrames for each set
    train_data = []
    validation_data = []
    test_data = []

    # Iterate through labeled CSV files
    for file in os.listdir('.'):
        if file.startswith('labeled_features1_session') and file.endswith('.csv'):
            # Extract session number from file name
            session_num = int(file.split('session')[-1].split('.')[0])

            # Read the CSV file
            df = pd.read_csv(file)

            # Append to the appropriate dataset based on session number
            if session_num in [1, 2, 3]:
                train_data.append(df)
            elif session_num == 4:
                validation_data.append(df)
            elif session_num == 5:
                test_data.append(df)

    # Combine all DataFrames for each set
    train_df = pd.concat(train_data, ignore_index=True)
    validation_df = pd.concat(validation_data, ignore_index=True)
    test_df = pd.concat(test_data, ignore_index=True)

    # Save the combined DataFrames to CSV files
    train_df.to_csv(output_train, index=False)
    validation_df.to_csv(output_val, index=False)
    test_df.to_csv(output_test, index=False)

    print(f"Training data saved to {output_train} ({len(train_df)} rows)")
    print(f"Validation data saved to {output_val} ({len(validation_df)} rows)")
    print(f"Test data saved to {output_test} ({len(test_df)} rows)")

    # Show the first 10 rows of each dataset
    print("\nTop 10 rows of the training dataset:")
    print(train_df.head(10))
    print("\nTop 10 rows of the validation dataset:")
    print(validation_df.head(10))
    print("\nTop 10 rows of the test dataset:")
    print(test_df.head(10))
```

```python
# Main function
def main():
    # Define output filenames
    output_train = "train_data1.csv"
    output_val = "validation_data1.csv"
    output_test = "test_data1.csv"

    # Combine CSV files into train, validation, and test datasets
    combine_csv_files(output_train, output_val, output_test)

if __name__ == "__main__":
    main()
```

Figure 3.6 Function to combine csv files into train, validation and test dataset

Finally, the **combine_csv_files() function** combines the labeled and normalized CSV files into training, validation, and test datasets. It reads the CSV files and appends the DataFrame to the appropriate dataset based on the session number. The combined DataFrames are then saved to new CSV files for training (session 1 to 3), validation (session 4) and testing set (session 5). This to maximize training data to learn and capture the features and the label.

Below is the output of after splitting the data into train, validation and test data.

Top 10 rows of the training dataset:

	file_name	frame_index	start_time	end_time	f0	mfcc_1	\
0	Ses01F_impro01.wav	0	0.00	0.05	-0.651306	3.535591	
1	Ses01F_impro01.wav	1	0.01	0.06	-0.651306	3.437276	
2	Ses01F_impro01.wav	2	0.02	0.07	-0.651306	2.844247	
3	Ses01F_impro01.wav	3	0.03	0.08	-0.651306	2.642039	
4	Ses01F_impro01.wav	4	0.04	0.09	-0.651306	2.516589	
5	Ses01F_impro01.wav	5	0.05	0.10	-0.651306	2.280618	
6	Ses01F_impro01.wav	6	0.06	0.11	-0.651306	2.067229	
7	Ses01F_impro01.wav	7	0.07	0.12	-0.651306	1.912242	
8	Ses01F_impro01.wav	8	0.08	0.13	-0.651306	1.837767	
9	Ses01F_impro01.wav	9	0.09	0.14	-0.651306	1.813043	

Top 10 rows of the validation dataset:

	file_name	frame_index	start_time	end_time	f0	mfcc_1	\
0	Ses04F_impro01.wav	0	0.00	0.05	-0.692658	3.946228	
1	Ses04F_impro01.wav	1	0.01	0.06	-0.692658	3.750927	
2	Ses04F_impro01.wav	2	0.02	0.07	-0.692658	3.012858	
3	Ses04F_impro01.wav	3	0.03	0.08	-0.692658	2.761822	
4	Ses04F_impro01.wav	4	0.04	0.09	-0.692658	2.620128	
5	Ses04F_impro01.wav	5	0.05	0.10	-0.692658	2.376961	
6	Ses04F_impro01.wav	6	0.06	0.11	-0.692658	2.258056	
7	Ses04F_impro01.wav	7	0.07	0.12	-0.692658	2.117117	
8	Ses04F_impro01.wav	8	0.08	0.13	-0.692658	1.954388	
9	Ses04F_impro01.wav	9	0.09	0.14	-0.692658	1.788543	

Top 10 rows of the test dataset:

	file_name	frame_index	start_time	end_time	f0	mfcc_1	\
0	Ses05F_impro01.wav	0	0.00	0.05	-0.596931	4.634518	
1	Ses05F_impro01.wav	1	0.01	0.06	-0.596931	4.489828	
2	Ses05F_impro01.wav	2	0.02	0.07	-0.596931	3.684195	
3	Ses05F_impro01.wav	3	0.03	0.08	-0.596931	3.370065	
4	Ses05F_impro01.wav	4	0.04	0.09	-0.596931	3.126479	
5	Ses05F_impro01.wav	5	0.05	0.10	-0.596931	2.914973	
6	Ses05F_impro01.wav	6	0.06	0.11	-0.596931	2.780298	
7	Ses05F_impro01.wav	7	0.07	0.12	-0.596931	2.589078	
8	Ses05F_impro01.wav	8	0.08	0.13	-0.596931	2.359004	
9	Ses05F_impro01.wav	9	0.09	0.14	-0.596931	2.163143	

Figure 3.7 Sample Output of training, validation and testing from Experiment 1

```
Training data saved to train_data1.csv (2452302 rows)
Validation data saved to validation_data1.csv (834063 rows)
Test data saved to test_data1.csv (879281 rows)
```

```
Training data saved to train_data2.csv (2452302 rows)
Validation data saved to validation_data2.csv (834063 rows)
Test data saved to test_data2.csv (879281 rows)
```

Figure 3.8 Number of rows in each csv files for Experiment 1 & Experiment 2

4.0 MODEL DEVELOPMENT

Here is the section where we explain our model selection for training the model to classify speaker changes. To train our model, we followed a structured approach. Initially, we trained the model without addressing the class imbalance in the dataset. In the second iteration, we implemented techniques to handle the imbalance. Finally, we experimented with different sliding window sizes. We then compared and evaluated the model's performance using two data extraction experiments for features F0 and MFCC, as described in Section 2.0: (1) **a 20ms window with a 10ms overlap** and (2) **an 80ms window with a 10ms overlap.**

4.1 Perform model LSTM on imbalance dataset

We have chosen to develop a model using a sequence-based LSTM (Long Short Term Memory) neural network for detecting speaker changes in audio data. Below are the steps in LSTM modelling:

- Load and prepare the data

```
def load_data(file_path):
    # Load data from CSV
    df = pd.read_csv(file_path)
    features = ['mfcc_1', 'mfcc_2', 'mfcc_3', 'mfcc_4', 'mfcc_5',
                'mfcc_6', 'mfcc_7', 'mfcc_8', 'mfcc_9', 'mfcc_10',
                'mfcc_11', 'mfcc_12', 'mfcc_13', 'f0']
    X = df[features].values
    y = df['speaker_change'].values
    return X, y

def prepare_data(X, y):
    # Reshape for LSTM input
    X = X.reshape((X.shape[0], 1, X.shape[1]))
    return X, y
```

Figure 4.1.1 Function to load data and prepare data

The load_data() function reads a CSV file, extracts the MFCC and F0 features as input,X and the speaker_change labels as output, y. Next, the prepare_data() function reshapes X into a 3D format required by LSTM models, (number of samples, timesteps=1, features).

- Create LSTM model

```python
def create_lstm_model(input_shape):
    model = Sequential([
        LSTM(64, input_shape=input_shape, return_sequences=True),
        Dropout(0.3),
        LSTM(32),
        Dropout(0.3),
        Dense(16, activation='relu'),
        Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer=Adam(learning_rate=0.0001),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model
```

Figure 4.1.2 Function to create lstm model

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_2 (LSTM)               (None, 1, 64)             20224

 dropout_2 (Dropout)         (None, 1, 64)             0

 lstm_3 (LSTM)               (None, 32)                12416

 dropout_3 (Dropout)         (None, 32)                0

 dense_2 (Dense)             (None, 16)                528

 dense_3 (Dense)             (None, 1)                 17

=================================================================
Total params: 33,185
Trainable params: 33,185
Non-trainable params: 0
```

Figure 4.1.3 Model Architecture

The two stacked LSTM layers help capture both short-term and long-term patterns in the sequence data. The create_lstm_model() function builds an LSTM-based neural network for binary classification. It includes two LSTM layers (64 and 32 units), each followed by dropout for regularization. A dense layer with 16 units (ReLU activation) extracts features, and a final sigmoid-activated output layer predicts the binary outcome. The model is compiled with the Adam optimizer, binary cross-entropy loss, and accuracy as the evaluation metric, making it ready for training.

The model summary is shown in Figure 4.1.3 with 33,185 number of trainable parameters, refers to the total number of parameters (weights and biases) in the model that will be learned during training. The model adjusts during training to minimize the loss function and improve performance.

- Define train model function

```python
def train_model(model, X_train, y_train, X_val, y_val):

    # Callbacks for early stopping and model checkpointing
    early_stop = EarlyStopping(
        monitor='val_loss',
        patience=10,
        restore_best_weights=True
    )

    # Train the model with class weights
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=20,
        batch_size=128,
        callbacks=[early_stop]
    )
    return history
```

Figure 4.1.4 Function to train model

This function trains the LSTM model using the training and validation data. It sets up early stopping to prevent overfitting and trains the model using the The function returns the training history for further analysis, allowing for model evaluation to make any necessary adjustment. The parameter that we set for this large and imbalanced dataset is batch_size = 128, patience =10, epoch=20, learning_rate=0.001.

4.2 Handling imbalanced dataset

```python
def balance_dataset(df):
    # Separate majority and minority classes
    df_majority = df[df['speaker_change'] == 0]
    df_minority = df[df['speaker_change'] == 1]

    # Determine the number of samples
    n_samples = min(len(df_majority), len(df_minority))

    # Undersample the majority class
    df_majority_undersampled = resample(df_majority,
                                        replace=False,    # sample without replacement
                                        n_samples=n_samples,
                                        random_state=42)  # reproducible results

    # Undersample the minority class (if needed)
    df_minority_undersampled = resample(df_minority,
                                        replace=False,    # sample without replacement
                                        n_samples=n_samples,
                                        random_state=42)  # reproducible results

    # Combine the balanced datasets
    df_balanced = pd.concat([df_majority_undersampled, df_minority_undersampled])

    # Shuffle the balanced dataset
    df_balanced = df_balanced.sample(frac=1, random_state=42).reset_index(drop=True)

    return df_balanced
```

Figure 4.2.1 Code snippets to balance the dataset

Here, we want to handle the imbalance dataset in training, validation and testing dataset obtained from assigning labels in Section 3.0. Handling this class imbalance is crucial to ensure that the model effectively learns to differentiate between the F0 and MFCC features associated with the speaker_change labels 0 (no speaker change) and 1 (speaker change). Below is the function to handle balancing the dataset. It ensures that the number of instances labeled as 1 in the training, validation, and testing datasets is determined and an equal number of instances labeled as 0 is selected to achieve a balanced dataset. So, when both classes have a similar number of records present in the dataset, the classifier will give equal importance to both classes.

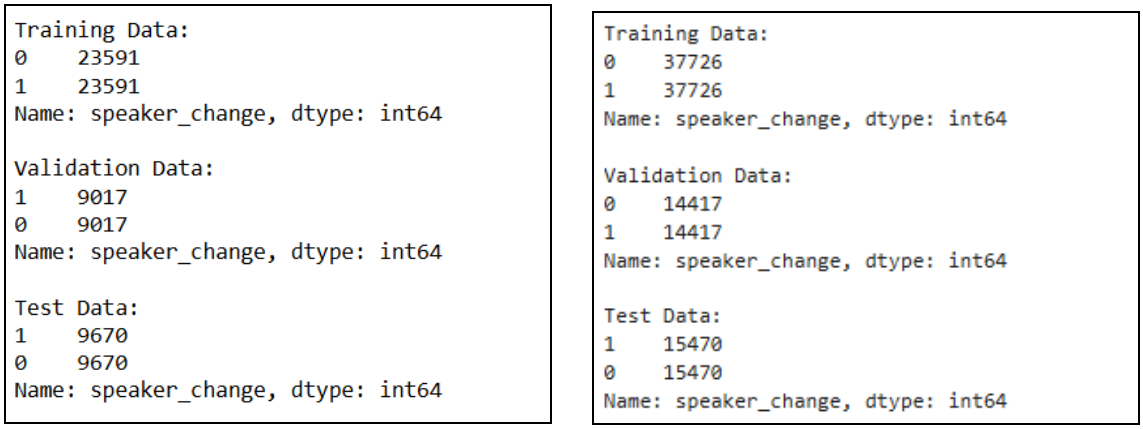


Figure 4.2.2 Class distribution for dataset after doing sampling
(Left: class distribution for Experiment 1, Right: class distribution for Experiment 2)

Experiment one implements 20ms sliding window while experiment 2 implements 80ms sized. The increasing size of the sliding window has increased the number of rows of the total dataset. This larger window allows for more detailed and accurate detection of speaker change, as it provides a broader context for each frame.

Then, we proceed to use the similar architecture LSTM model for a balanced dataset. However, we just changed some parameters of the model to adjust that suit with a lower number of dataset after doing balancing class distribution. So the parameter changed is the batch_size is reduced to 64 and patience changed to 4.

4.3 Experiment with another dataset of different window size

The first experiment of a balanced dataset was using 20ms frame length or called sliding window size with 10ms frame shift. After experimenting with this window, we performed the model development with a different dataset configuration which is an 80ms window with a 10ms overlap. This adjustment which increased the size frame aimed to improve the model performances as greater window size provides a more detailed sampling to be done. Additionally, the total number of rows in the dataset increased, thus contributing to a more robust training, validation, and testing processes. The result performance and plot of training and accuracy loss is discussed in Section 5.0.

5.0 RESULT AND DISCUSSION

5.1 Test the model on unseen data

Function **evaluate_model(model, X_test, y_test)** takes parameters like *model, X_test,* and *y_test* as the unseen data. This function firstly predicts binary values based on threshold 0.5. Then, this function prints a classification report, which includes metrics like precision, recall, and F1-score, providing a detailed evaluation of the model's performance. Next, this function will calculate the confusion matrix, which shows the number of *True Positive, True Negative, False Positive* and *False Negative.* Heatmap used to visualize this matrix as displayed in figure 5.2.1 and figure 5.2.2 to make an easy interpretation of the result.

```
def evaluate_model(model, X_test, y_test):
    # Predict and evaluate
    y_pred = (model.predict(X_test) > 0.5).astype(int)

    print("Classification Report:")
    print(classification_report(y_test, y_pred))

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title('Confusion Matrix')
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.show()
    plt.tight_layout()
    plt.savefig('confusion_matrix.png')
    plt.close()
```

Figure 5.1.1 Code snippet of evaluate function model

5.2 Comparison performance on test data between 3 model of different window sizes with balanced or imbalanced dataset

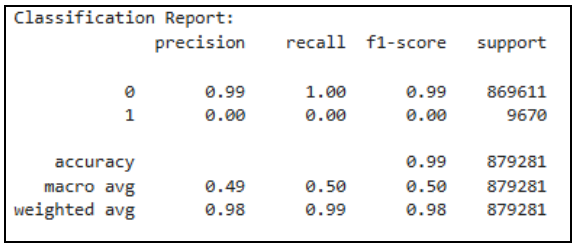
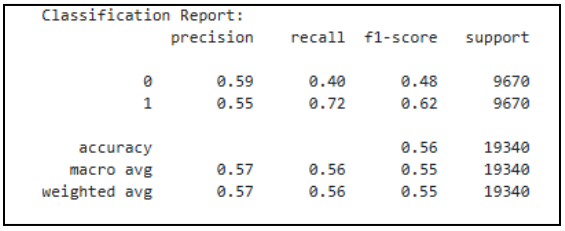
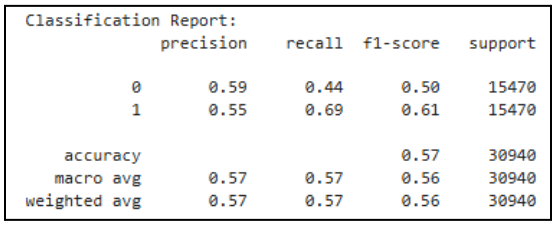
<table>
<tr>
<td>Experiment 1 (sliding window 20 with 10ms overlap) + No Balanced Dataset</td>
<td>

Classification Report:
precision recall f1-score support
0 0.99 1.00 0.99 869611
1 0.00 0.00 0.00 9670
accuracy 0.99 879281
macro avg 0.49 0.50 0.50 879281
weighted avg 0.98 0.99 0.98 879281
Figure 4.4.1 Classification Report of Experiment 1 (Using Not Balanced dataset)
</td>
</tr>
<tr>
<td>Experiment 1 (sliding window 20 with 10ms overlap) + Balanced Dataset</td>
<td>

Classification Report:
precision recall f1-score support
0 0.59 0.40 0.48 9670
1 0.55 0.72 0.62 9670
accuracy 0.56 19340
macro avg 0.57 0.56 0.55 19340
weighted avg 0.57 0.56 0.55 19340
Figure 4.4.1 Classification Report of Experiment 1 (Using balanced dataset)
</td>
</tr>
<tr>
<td>Experiment 2 (sliding window 80 with 10ms overlap) + Balanced Dataset</td>
<td>

Classification Report:
precision recall f1-score support
0 0.59 0.44 0.50 15470
1 0.55 0.69 0.61 15470
accuracy 0.57 30940
macro avg 0.57 0.57 0.56 30940
weighted avg 0.57 0.57 0.56 30940
Figure 4.4.1 Classification Report of Experiment 2 (Using balanced dataset)
</td>
</tr>
</table>

Table 5.2.1 Table Comparison of Classification Report

Table above is the comparison classification report between 3 experiments. Below is the explanation for each classification report.

A. Experiment 1 (sliding window 20 with 10ms overlap) + No Balanced Dataset

- This experiment is an imbalanced dataset (869,602 vs 9,670 samples) with a model that's performing poorly on the minority class (label 1). While the overall accuracy is high at 99%, this is misleading because the model is failing to identify any class 1 instances (0% recall). The recall for class 0 is 1.00, which means it predicted all instances as 0. This is due to problems in imbalanced datasets where the model learns to predict the majority class well but fails on the minority class.
- Thus, even if the accuracy is high, the model fails to predict the speaker change (label 1) and this model cannot be compared using accuracy evaluation.

B. Experiment 1 (sliding window 20 with 10ms overlap) + Balanced Dataset

- For class 0, the model achieves a precision of 59%, meaning that when it predicts an instance as class 0, it is correct 59% of the time. However, its recall for class 0 is lower at 40%, indicating it misses more than half of the actual class 0 instances. The F1-Score is 0.48.
- For class 1, the model has a precision of 0.55, meaning that 55% of the instances predicted as class 1 are actually class 1. The recall is 0.72, indicating that the model is more successful at identifying class 1 instances compared to the imbalance dataset above.
- The model's accuracy is 0.56, meaning that 56% of the total predictions are correct.

C. Experiment 2 (sliding window 80 with 10ms overlap) + Balanced Dataset

- For class 0, the model achieves a precision of 59%, same as Experiment 1 above, meaning that when it predicts an instance as class 0, it is correct 59% of the time. The model has slightly higher recall of 0.44 and F1-score of 0.50 than Experiment 1.This may be because this dataset has a higher number of rows 0 than the dataset in Experiment 1.
- For class 1, the model has a precision of 0.55, same as Experiment 1 above, meaning that when it predicts an instance as class 1, it is correct 55% of the time. The recall is 0.69 and the F1-score is 0.61 slightly lower than experiment 1. This may be because of a larger number of rows 1 than the dataset in Experiment 1.
- The model's accuracy is 57%, which improves 1% better than the previous dataset in Experiment 1 of sliding window 20ms.

Summary Comparison Performance

1. Comparison based on imbalanced dataset in A and balanced dataset in B
    - The model in B performs better even though the accuracy of B, 56% is lower than 99% of model A. The model B is making genuine attempts to identify both classes, unlike the model A where it only predicted class 0.
    - The performance of model B can be evaluated using recall and F1-score. The model is better in predicting class 1 (speaker change) than class 0 (no speaker change) as indicated by the higher recall and F1-score for class 1 compared to the imbalanced dataset in A.

2. Comparison based on different sliding window in model B and model C
    - This performance comparison can be evaluated using accuracy as both already have handling the imbalance dataset but they differ in sliding window. The model C has higher accuracy of 1% compared to model B, which we assume by using a bigger sliding window, model C can provide more context for classification than model B.

Discussions on Performance of Sliding Window 20ms and 80ms

Here we want to discuss the assumptions why the accuracy improvement in model C is minimal (just increase by 1%) despite using a larger window size.

1. Information Saturation:

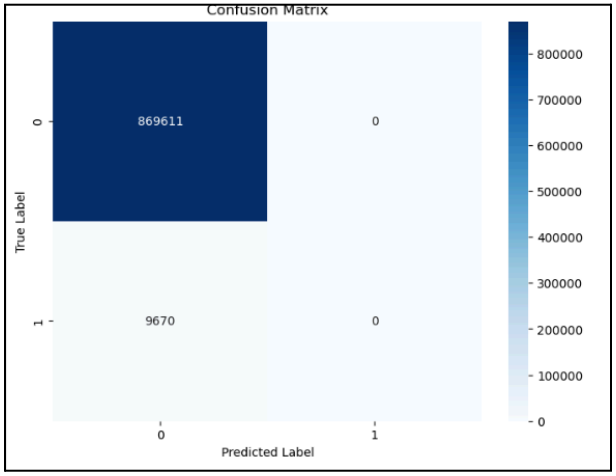
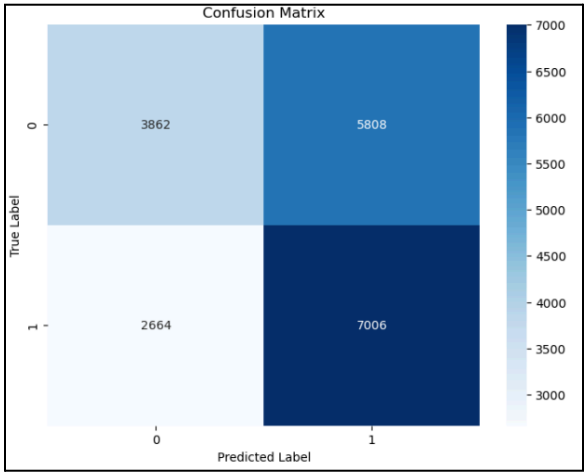
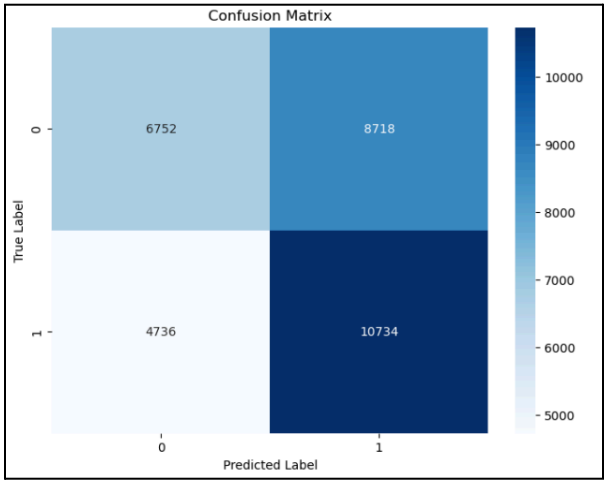
    The larger window (80) might be capturing redundant information compared to the smaller window (20). The additional temporal data may not provide significantly new or discriminative features for classification. Also, the 10ms overlap in both cases might already be capturing the essential patterns.

2. Feature Relevance:

    The discriminative features for classification might be better captured in shorter time segments. The larger window could be introducing noise or less relevant information. The critical patterns which are the MFCC and F0 features for classification might be localized in shorter time spans.

3. Signal Characteristics:

    The underlying signal patterns might be better represented in smaller windows. Temporal dependencies might be more meaningful in shorter sequences. The signal's distinctive features might get diluted in longer windows.

Experiment 1 (sliding window 20 with 10ms overlap) + No Balanced Dataset	Experiment 1 (sliding window 20 with 10ms overlap) + Balanced Dataset	Experiment 2 (sliding window 80 with 10ms overlap) + Balanced Dataset
 *Figure 5.2.1 Confusion matrix from Experiment 1 with Imbalanced Dataset*	 *Figure 5.2.2 Confusion matrix from Experiment 1*	 *Figure 5.2.3 Confusion matrix from Experiment 2*

A) Class Imbalance: The experiments conducted have shown the importance of considering balancing the dataset. Experiment 1 with an imbalance dataset suffered extreme class imbalance where the coefficient matrix predicted biasedly.

B) Model Performance: The experiments conducted with a balanced dataset show a similar pattern of its confusion matrices. This means, the experiments with balanced dataset are better at detecting positive cases which is class 1.

C) Scale Differences: Experiment 2 conducted with more dataset which roughly 1.6 times more samples compared to experiment 1. This is because The greater the sliding window size, the greater the number of times the window capture data points.

Table 5.2.2 Comparison on the Confusion Matrix of Different Experiments

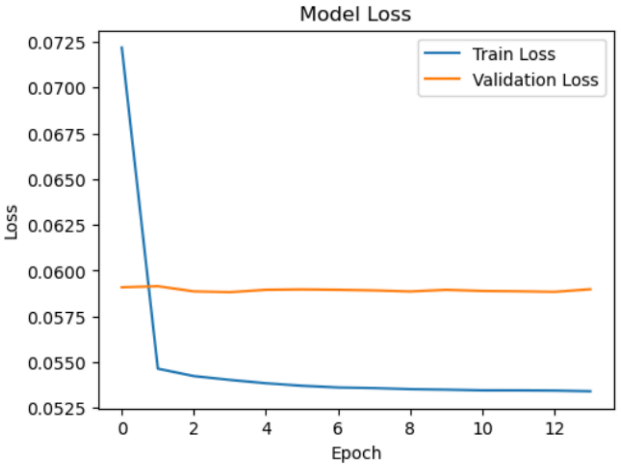
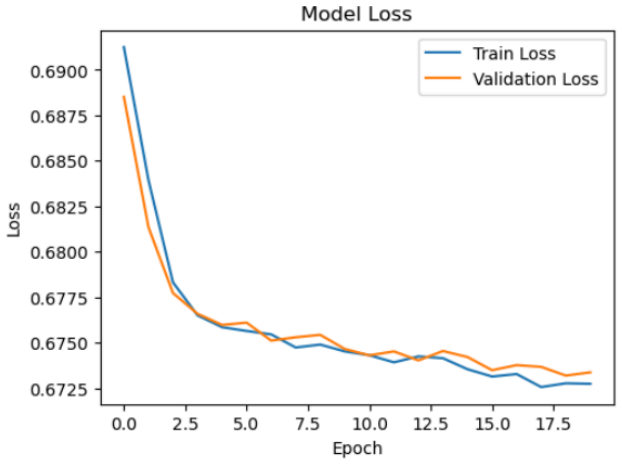
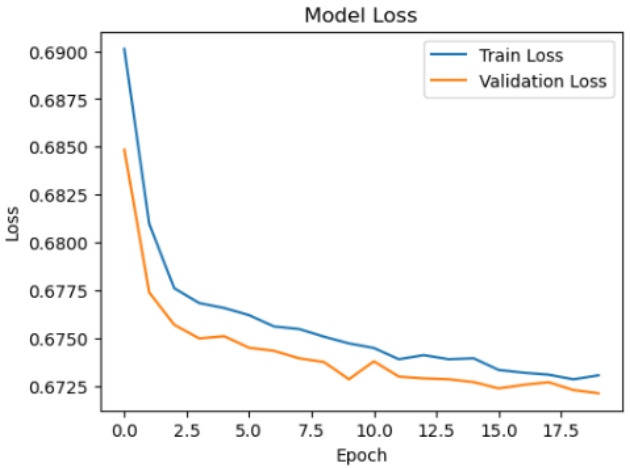
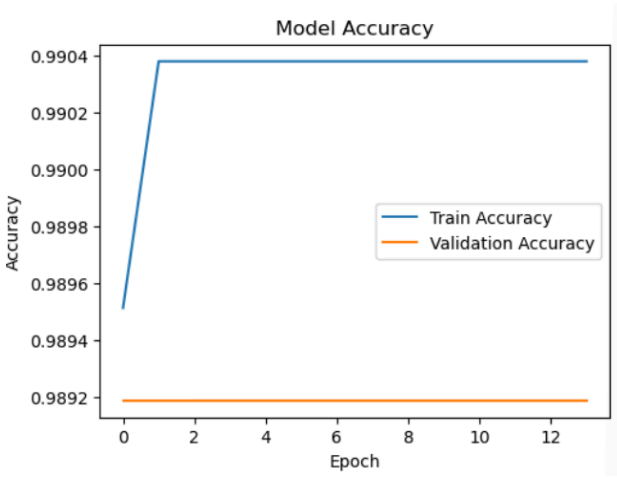
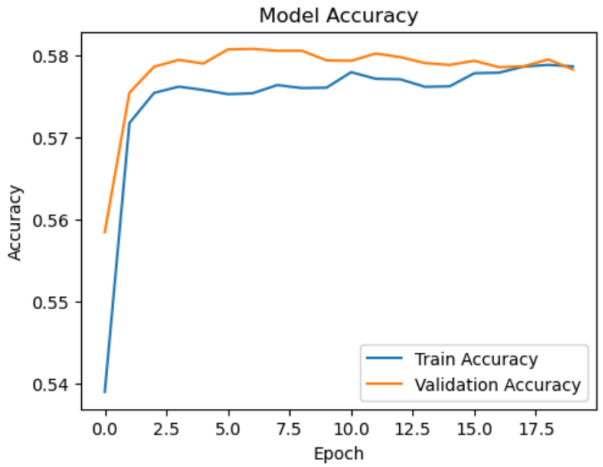
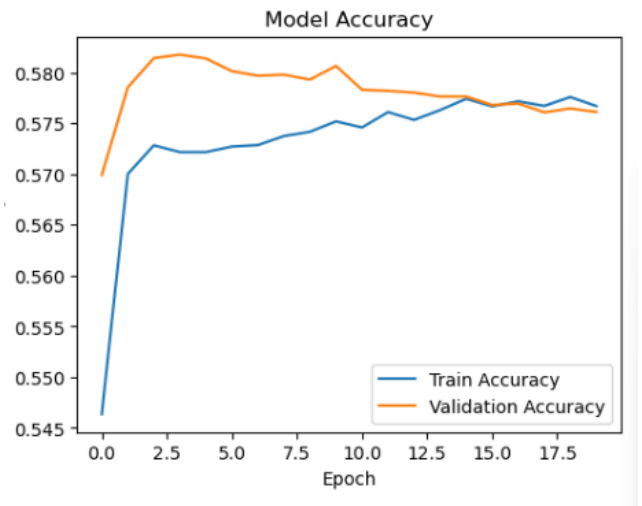
Experiment 1 (sliding window 20 with 10ms overlap) + No Balanced Dataset	Experiment 1 (sliding window 20 with 10ms overlap) + Balanced Dataset	Experiment 2 (sliding window 80 with 10ms overlap) + Balanced Dataset
 *Figure 5.2.4 Model Loss for Experiment 1 with Imbalanced Dataset*	 *Figure 5.2.5 Model Loss for Experiment 1*	 *Figure 5.2.6 Model Loss for Experiment 2*
- The training loss decreased over the epoch making gaps away from the validation loss where it recorded a constant loss on top of that. - Final loss values recorded around 0.0525 until 0.06	- Both training and evaluation pattern decrease smoothly - The training and validation loss are so close indicates that this model has a better generalization - Training loss slightly higher compare to evaluation loss, lower after epoch 2 - FInal loss values around 0.6725 - 0.6775	- Both training and evaluation pattern decrease smoothly - This graph shows a more separation of both training and validation loss - The training loss recorded slightly higher compare to evaluation loss over the epoch - Final loss values around 0.6725 - 0.6750

Table 5.2.2 Comparison on the Model Loss of Different Experiments

Experiment 1 (sliding window 20 with 10ms overlap) + No Balanced Dataset	Experiment 1 (sliding window 20 with 10ms overlap) + Balanced Dataset	Experiment 2 (sliding window 80 with 10ms overlap) + Balanced Dataset
		
Figure 5.2.7 Model Accuracy for Experiment 1 with Imbalanced Dataset	*Figure 5.2.8 Model Accuracy for Experiment 1*	*Figure 5.2.9 Model Accuracy for Experiment 2*

- The experiments that were conducted using a balanced dataset recorded model accuracy around 57% - 59%. Both training and validation metrics track closely in these experiments. This illustrates these experiments having healthy learning with good generalization. Moreover, this experienced gradual improvement over epochs. This demonstrates the learning process in training and validation is consistent and balanced.
- On the other hand, the experiments with an imbalanced dataset recorded an extremely higher accuracy which is approximately 99%. The experiment conducted with an imbalanced dataset demonstrates a clear overfitting pattern. This is due to the training accuracy which is around 99% and the validation accuracy that stays flat around 98%, creating a large gap between training and validation performance.

Table 5.2.2 Comparison on the Accuracy Loss of Different Experiments

5.3 Analyze model performance at detecting speaker change points, discussing strengths and weaknesses. suggestions for improvement.

Initially, the model faced severe class imbalance where the model prediction was heavily skewed towards class 0, which means the model completely failed to detect any speaker changes which is the class. While the coefficient matrix initially showed a positive result, relying solely on this matrix could cause potential misleading. The high accuracy was due to the small ratio of class 1 to class 0 instances. Thus handling an imbalance dataset is needed at this point. After implementing techniques to handle the imbalance dataset, the overall accuracy decreased, to 56% to 59%. This reduction was expected to no longer bias towards the majority class, thus could make a genuine attempt to identify instances of both classes.

Aspects	Strengths	Weaknesses
Sliding Window Size (Frame Size)	Increasing the sliding window size gives chances to capture broader temporal context. This can enhance the detection performance, thus leading to more accurate and reliable analysis.	Increasing the sliding window size makes the computational load and the amount of data to be loaded greater. To perform training with higher frame size, more resources and time required.
Overlap between Sliding Windows (Frame Shift)	Reducing the overlap between sliding windows decreases the number of times each data included in multiple windows as well as the number of rows of the extracted dataset. The good thing is, this approach simplifies the computation by reducing resources and time to perform analysis.	However, the reduction of frame shifts will also reduce the contextual information captured, throughout the process. This will affect the accuracy and the reliability as not so much detailed information is gathered.
Handling Class Imbalance	The model performance improved drastically by addressing the extreme class imbalance. This adjustment enables the model to focus on the minority class.	The process of undersampling of the majority class may led to loss meaningful information
Batch Size	Considering batch size ensures balance between computational efficiency and model performance. Smaller batches helped with fine-tuning the gradients, while larger batches improved training stability.	While batch size was optimized for computational efficiency, its effect on model convergence and generalization was not deeply analyzed. Larger batch sizes might stabilize the gradients further for balanced datasets.
Model Selection LSTM	LSTM excel at capturing temporal dependencies in time-series data. It can automatically learn relevant features from raw sequential data and capture complex non-linear relationships. Lastly, LSTM is effective in learning hierarchical patterns across different time scales.	LSTM drawback is longer training times due to sequential processing. It requires significant amounts of data for effective learning and also sensitive to hyperparameter selection (learning rate, number of layers, units). Lastly,it prones to overfitting without proper regularization.

Suggestions for Improvement

Here we provide our improvements that can be done for future work in speaker change detection:

1. Window Configuration Optimization:

    - When adjusting the sliding window size and amount of overlap, it is important to consider various factors or contexts that are suitable to the dataset to achieve the optimal performance of the model.
    - For example, explore varying frame sizes from 20ms to 100ms to find optimal balance between temporal resolution and context
    - Test different overlap ranges (5ms to 15ms) to evaluate impact on detection accuracy
    - Systematically analyze the relationship between window parameters and model performance

2. Class Imbalance Solutions:

    - Trying different alternatives and approaches to address class imbalance issues such as losing valuable data from undersampling.
    - For instance, implement advanced sampling techniques beyond basic undersampling
    - Explore synthetic data generation methods like SMOTE to balance the dataset without losing valuable information
    - Consider hybrid approaches combining sampling techniques with algorithmic solutions

3. Model Architecture and Training:
    - Experiment modelling with different batch sizes (32, 64, 128) to optimize training dynamics
    - Analyze the impact of batch size on gradient stability and convergence speed
    - Perform extensive hyperparameter tuning using techniques like grid search or random search to find the optimal set of hyperparameters for the model.
    - Experiment with more advanced neural network architectures such as Bidirectional LSTM, GRU (Gated Recurrent Unit), or Transformer models. These architectures may capture temporal dependencies more effectively.
    - Integrate attention mechanisms to allow the model to focus on important parts of the audio sequence, potentially improving performance.

4. Feature Engineering and Analysis:
    - Exploring additional acoustic features that might improve change detection such as pitch, energy, and formants.
    - Study the impact of different feature combinations on model performance
    - Consider domain-specific feature extraction methods

5. Data Augmentation
    - Generate synthetic data to augment the training dataset. This can help address class imbalance and improve the model's generalization capabilities.

6. Transfer Learning
    - Leverage pre-trained models on similar tasks and fine-tune them on the speaker change detection dataset. This can help improve performance with limited training data.

6.0 CONCLUSION

In a nutshell, this assignment demonstrated a thorough exploration of speaker change detection using LSTM-based neural networks. The model showcased the potential of sequence-based architectures for this task by leveraging Fundamental Frequency (F0) and Mel-Frequency Cepstral Coefficients (MFCC) extracted from the audio dataset. The experiments conducted emphasize the importance of addressing class imbalance in the dataset and fine-tuning model parameters, such as sliding window sizes, to optimize performance.

Through this assignment, we learned that sequence models like LSTMs are highly effective for tasks involving temporal dependencies, such as speaker change detection. The choice of features (F0 and MFCC) plays a critical role in capturing meaningful patterns from the audio data. Additionally, handling dataset imbalances significantly improves model accuracy and robustness, highlighting the need for thoughtful preprocessing. Experimenting with different configurations, such as varying sliding window sizes, demonstrates how parameter tuning directly impacts model performance.

Overall, this assignment provided valuable insights into building and evaluating models for speaker change detection, reinforcing the importance of proper feature extraction, handling data imbalances, and systematically experimenting to achieve optimal results.

7.0 REFERENCES

1. Mazumder, S. (2021, June 21). *What is Imbalanced Data | Techniques to Handle Imbalanced Data*. Analytics Vidhya. https://www.analyticsvidhya.com/blog/2021/06/5-techniques-to-handle-imbalanced-data-for-a-classification-problem/
2. mrt. (2017, August 22). *What data rescaling preprocessing do I need to do before training an LSTM NN?* Stack Overflow. https://stackoverflow.com/questions/45822254/what-data-rescaling-preprocessing-do-i-need-to-do-before-training-an-lstm-nn
3. Brownlee, J. (2016, July 25). *Sequence Classification with LSTM Recurrent Neural Networks in Python with Keras*. Machine Learning Mastery. https://machinelearningmastery.com/sequence-classification-lstm-recurrent-neural-networks-python-keras/
4. *LSTM Networks*. (2022). Apmonitor.com. https://apmonitor.com/do/index.php/Main/LSTMNetwork
5. J, R. T. J. (2021, September 10). *LSTMs Explained: A Complete, Technically Accurate, Conceptual Guide with Keras*. Analytics Vidhya. https://medium.com/analytics-vidhya/lstms-explained-a-complete-technically-accurate-conceptual-guide-with-keras-2a650327e8f2
6. *Determining whether a dataset is imbalanced or not*. (n.d.). Data Science Stack Exchange. https://datascience.stackexchange.com/questions/122571/determining-whether-a-dataset-is-imbalanced-or-not
7. Devansh. (2024, February 28). *Mastering the Sliding Window Technique - Devansh - Medium*. Medium.

https://machine-learning-made-simple.medium.com/mastering-the-sliding-window-technique-db8dcb108376

8. Brownlee, J. (2019, April 26). *Time Series Prediction with LSTM Recurrent Neural Networks in Python with Keras*. Machine Learning Mastery. https://machinelearningmastery.com/time-series-prediction-lstm-recurrent-neural-networks-python-keras/
9. Bang, J., Hur, T., Kim, D., Huynh-The, T., Lee, J., Han, Y., Banos, O., Kim, J.-I., & Lee, S. (2018). Adaptive Data Boosting Technique for Robust Personalized Speech Emotion in Emotionally-Imbalanced Small-Sample Environments. *Sensors*, *18*(11), 3744. https://doi.org/10.3390/s18113744